

AI Resume Analyzer & ATS Checker

Project Overview: Modules, Technology Stack & Output

1. Introduction (Idea)

AI Resume Analyzer & ATS Checker is a web application where a user uploads their resume and, optionally, a job description, and receives an AI-generated analysis of the resume's quality along with an ATS (Applicant Tracking System) compatibility score. The AI, powered by the Groq API, reviews formatting, keywords, structure and content, then gives clear, actionable suggestions to improve the resume's chances of passing automated screening and impressing a recruiter.

2. Objectives

- Let users upload a resume and get an instant AI-based quality analysis.
- Score how well a resume is likely to pass an ATS, out of 100.
- Compare a resume against a specific job description and highlight missing keywords.
- Give clear, actionable suggestions to improve wording, structure and formatting.
- Keep a history of past resume analyses so users can track improvement over time.

3. User Roles

Role	Main Responsibilities
User	Registers, uploads resumes, runs ATS checks against job descriptions, views feedback and history.
Admin	Manages users, reviews usage statistics, updates scoring rules and keyword rules.

4. System Modules

The system consists of the following modules, all managed by the FastAPI backend.

Module	Description	Key Features
1. Login & Authentication	Secure access for users and admin.	Register/login, JWT tokens, password hashing, role-based access, all handled by the backend.
2. Resume Upload	Brings in the resume file.	Upload PDF/DOCX resumes, file type and size validation, text extraction.
3. Job Description Input	Captures the target role.	Paste or upload a job description to compare the resume against a specific role.
4. Resume Parsing	Extracts structured resume data.	Extracts name, contact, sections, skills, experience and education from the raw text.

Module	Description	Key Features
5. ATS Score Checker	Scores ATS compatibility.	Checks formatting (tables, columns, fonts, headers), file type and section structure; gives a score out of 100.
6. AI Resume Analysis (Groq)	Reviews content quality.	AI reviews clarity, achievement-based wording, grammar and overall impact; explains strengths and weaknesses.
7. Keyword & Skill Matching	Compares to the job description.	Finds keywords present in the job description but missing from the resume; suggests where to add them.
8. Improvement Suggestions	Turns feedback into action.	AI-generated, section-by-section suggestions (summary, experience, skills) to raise the score.
9. Analysis History	Tracks progress over time.	Saved past analyses and scores so a user can see improvement across resume versions.
10. Admin Dashboard & Reports	Supports oversight.	Total analyses run, average ATS scores, common missing keywords; export to PDF/Excel.

5. Analysis Workflow

1. User uploads resume	2. Job description added (optional)	3. Resume parsed & checked	4. Groq AI analyses content	5. Score, feedback & tips shown
------------------------	-------------------------------------	----------------------------	-----------------------------	---------------------------------

6. ATS Scoring & AI Approach

Concept	How It Works Here
Text Extraction	PyPDF2/pdfplumber (PDF) or python-docx (Word) extract raw text from the uploaded resume, run in the backend.
Rule-Based ATS Checks	Deterministic checks: standard fonts, no tables/images for key content, clear section headers, contact info present, correct file type.
Keyword Matching	Keywords extracted from the job description (via simple NLP) are compared against resume text to find gaps.
LLM via Groq API	The backend sends the resume text (and job description, if given) to a Groq-hosted large language model for qualitative review and suggestions.
System Prompt	A fixed prompt instructs the model to act as a resume/ATS expert: score consistently, explain reasoning briefly, and avoid inventing facts not in the resume.
Key Handling	The Groq API key lives only in the backend's .env file. The Next.js frontend never sees it and calls only the FastAPI endpoints.

7. System Architecture

The system follows a three-tier architecture with a Groq AI service called by the backend:

Frontend Next.js with reusable UI components (upload form, score cards, report view)	Backend Python FastAPI: auth, database access, parsing, ATS rules and Groq API calls	Database MongoDB (backend-managed)	AI Service Groq API (LLM), called only from the backend
--	--	--	---

The Next.js frontend uploads the resume to the FastAPI backend over REST. The backend validates the JWT, extracts and parses the resume, runs the rule-based ATS checks, calls the Groq API for qualitative analysis, saves the result to MongoDB and returns the score and feedback as JSON. The frontend has no direct database, auth or AI-key access.

8. Main Database Collections (MongoDB)

users, resumes, job_descriptions, ats_scores, analyses, suggestions, keyword_gaps, analysis_history, admin_settings.

9. Technology Stack

Layer	Technology (Primary)	Alternatives
Frontend	Next.js (React) with TypeScript	React (Vite), Vue / Nuxt
UI Components	Reusable Next.js components: shadcn/ui or Material UI with Tailwind CSS; upload dropzone, score gauge, report cards	Ant Design, Chakra UI
Backend	Python FastAPI (async REST APIs); owns auth, database access, parsing and AI calls	Django REST Framework, Node.js (Express)
Database	MongoDB (document database)	PostgreSQL
Database Access	Motor (async driver) with Pydantic models; or Beanie ODM	PyMongo, Prisma
Resume Parsing	PyPDF2/pdfplumber (PDF), python-docx (Word)	Apache Tika
AI Analysis	Groq API (LLM, e.g. Llama 3) via the official Groq Python SDK; streaming responses	OpenAI API, Gemini, local LLM (Ollama)
Keyword Matching	spaCy or scikit-learn (TF-IDF) for extracting and comparing keywords	RAKE, KeyBERT
Authentication	JWT issued and verified in FastAPI; password hashing (passlib/bcrypt)	OAuth 2.0 / Google login
Secrets	API keys and DB credentials in the backend .env file, loaded with python-dotenv; never in frontend code or Git	Cloud secret manager
Tools	VS Code, Git & GitHub, Postman, MongoDB Compass	Insomnia, Docker Desktop
Deployment	Vercel for Next.js; Uvicorn/Gunicorn on Render, Railway or AWS; MongoDB Atlas	Docker, Azure

10. Output Recommendation

The system produces the following outputs and screens for its users.

Output	What It Shows
ATS Score Card	Overall ATS compatibility score out of 100, with a pass/needs-work indicator.
AI Analysis Report	Groq-generated feedback on clarity, impact and structure, with strengths and weaknesses.
Keyword Gap List	Keywords from the job description that are missing from the resume.
Section-by-Section Suggestions	Specific rewrite tips for the summary, experience and skills sections.
Formatted Resume Preview	The parsed resume text/sections, so the user can confirm extraction was accurate.
Analysis History	Past resumes and scores, so the user can track improvement over time.
Admin Dashboard	Total analyses run, average scores and common missing keywords across users.

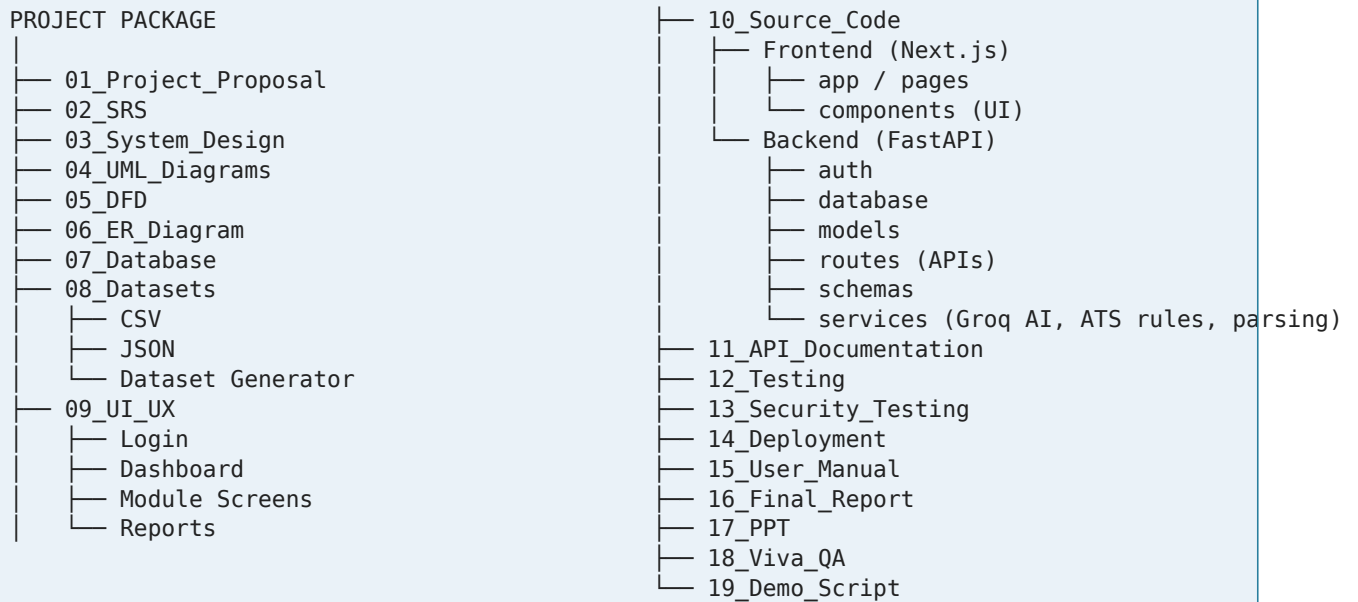
11. Benefits & Future Enhancements

Benefits	Future Enhancements
• Instant, objective feedback instead of guesswork. • Higher chance of passing automated ATS screening. • Clear, specific suggestions rather than vague tips. • Tracks improvement across resume versions.	• Resume rewriting assistant that edits text directly. • Cover letter analysis and generation. • LinkedIn profile optimisation suggestions. • Browser extension to check resumes against live job postings.

Note: API keys, database credentials and JWT secrets belong only in a local .env file (excluded from Git via .gitignore) or a secrets manager - never in code, chat, or this document.

12. Project Package Structure

All project deliverables are organised into one package of 19 folders, from planning documents to source code, testing and final presentation.



Folder	Contents
01_Project_Proposal	Problem statement, objectives, scope and feasibility.
02_SRS	Functional and non-functional requirements (SRS).
03_System_Design	Architecture, module design and technology choices.
04_UML_Diagrams	Use case, class, sequence and activity diagrams.
05_DFD	Data flow diagrams (Level 0, 1 and 2).
06_ER_Diagram	Diagram of the MongoDB collections and their relationships.
07_Database	MongoDB collection design, indexes and seed data scripts.
08_Datasets	Sample resumes, job descriptions and ATS scores in CSV and JSON, plus a dataset generator.
09_UI_UX	Screens for Login, Upload, ATS Score, Analysis Report and Dashboard.
10_Source_Code	Two folders only: Frontend (Next.js pages and UI components) and Backend (FastAPI). Backend holds auth (login, JWT), database (MongoDB), models, routes (REST APIs), schemas and services (Groq AI analysis, ATS rules, resume parsing).
11_API_Documentation	FastAPI auto-generated OpenAPI/Swagger docs for all endpoints.
12_Testing	Unit, integration and user acceptance tests, including ATS scoring and parsing test cases.
13_Security_Testing	Login, role-access, API key protection, file-upload validation, injection and XSS tests.
14_Deployment	Server setup, environment variables (.env) and deployment steps.
15_User_Manual	Step-by-step usage guide for users and admin.
16_Final_Report	Complete project report.
17_PPT	Presentation slides.
18_Viva_QA	Expected viva questions with answers.

Folder	Contents
19_Demo_Script	Step-by-step flow for the live project demo.